

CS543 Web Services — Assignment 2

CampusEats — Services, Contracts & Schema

Design benchmark derived from the Assignment 1 CampusEats concept: a campus food-ordering system for students and food vendors.

1. Capability List

#	Capability	What the system does
1	Browse vendors and menus	Students can discover campus food vendors and view available menu items.
2	Search and filter food	Students can search menu items and filter by category, price, and availability.
3	Manage cart	Students can add, remove, and change quantities of menu items before ordering.
4	Place an order	The system validates the cart, creates an order, calculates totals, and starts payment.
5	Pay for an order	The system authorizes and records payment for an order.
6	Track order status	Students can view order states such as PLACED, CONFIRMED, PREPARING, READY, and COMPLETE.
7	Manage menu	Vendors can create, update, disable, and price their menu items.
8	Process vendor orders	Vendors can accept orders and update preparation status.
9	Send notifications	The system sends order/payment/status notifications to students.

2. Service Boundaries and Data Ownership

Service	Data it owns
Identity Service	Student profiles, vendor accounts, authentication identities
Catalogue Service	Vendors, menus, menu items, categories, item availability
Order Service	Carts, orders, order items, order status
Payment Service	Payment attempts, transaction status, payment references
Notification Service	Notification records and delivery status

Boundary rule: a table belongs to exactly one service. Other services may hold only opaque IDs and use contracts to obtain or validate information.

3. Service Contracts

Identity Service

Operation	Input	Output	Errors
getStudent	studentId	StudentSummary	NOT_FOUND, UNAVAILABLE
validateStudent	studentId	ValidationResult	NOT_FOUND, SERVICE_UNAVAILABLE

Catalogue Service

Operation	Input	Output	Errors
searchItems	query, category, vendorId, page	ItemSummary[]	INVALID_QUERY, UNAVAILABLE
checkItems	items[{itemId, quantity}]	ItemCheckResult[]	ITEM_NOT_FOUND, ITEM_UNAVAILABLE, INVALID_
reserveItems	reservationId, items[]	ReservationResult	ITEM_UNAVAILABLE, RESERVATION_FAILED
releaseReservation	reservationId	ReleaseResult	RESERVATION_NOT_FOUND, UNAVAILABLE

Order Service

Operation	Input	Output	Errors
getOrder	orderId, studentId	OrderSummary	NOT_FOUND, FORBIDDEN
listStudentOrders	studentId, page	OrderSummary[]	INVALID_PAGE, UNAVAILABLE
placeOrder	studentId, items[], paymentMethod	PlaceOrderResult	VALIDATION_FAILED, ITEM_UNAVAILABLE, PAYME

Payment Service

Operation	Input	Output	Errors
authorizePayment	paymentId, studentId, amount, method	PaymentResult	DECLINED, INVALID_AMOUNT, PROVIDER_UNAVA
getPayment	paymentId	PaymentResult	NOT_FOUND, UNAVAILABLE

Notification Service

Operation	Input	Output	Errors
sendOrderNotification	studentId, orderId, event, message	NotificationResult	INVALID_EVENT, DELIVERY_FAILED
getNotifications	studentId, page	NotificationSummary[]	INVALID_PAGE, UNAVAILABLE

4. Full Central Operation — placeOrder

Purpose: Create a valid CampusEats order for a student and initiate payment without exposing internal storage, reservation, transaction, or payment-provider details.

Inputs

studentId; items [{itemId, quantity}]; paymentMethod. The caller does not supply prices, totals, vendor details, order status, payment transaction IDs, or database identifiers generated internally.

Output

PlaceOrderResult containing orderId, status, totalAmount, currency, and paymentStatus. Internal reservation IDs and payment-provider references are not exposed.

Error cases

VALIDATION_FAILED, STUDENT_NOT_FOUND, ITEM_NOT_FOUND, ITEM_UNAVAILABLE, INVALID_QUANTITY, RESERVATION_FAILED, PAYMENT_FAILED, and DEPENDENCY_UNAVAILABLE.

Internal flow

1. Validate the student through Identity Service.
2. Validate the request shape: non-empty items, positive quantities, and an accepted payment method.
3. Ask Catalogue Service to check item availability and obtain current item pricing.
4. Calculate the order total inside Order Service; the client never supplies the trusted price.
5. Ask Catalogue Service to reserve the requested quantities.
6. Create the order and order-item records with status PAYMENT_PENDING.
7. Ask Payment Service to authorize the calculated amount.
8. If payment succeeds, mark the order PLACED and confirm the reservation; if payment fails, cancel the order and release the reservation.
9. Ask Notification Service to send the order confirmation.
10. Return only the public result defined by the contract.

What is hidden: database tables, SQL, transaction boundaries, reservation implementation, payment-provider API, retry/idempotency logic, generated internal IDs, and notification delivery mechanism.

5. Service Validation

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled	Validation / fix
Identity Service	Yes	Yes	Yes	Yes	Yes	No issue; uses its own identity data and exposes stable student operations.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	No issue; owns all menu/catalogue data.
Order Service	Yes	Yes	Yes	Yes	Yes	No issue; cross-service IDs are opaque references, not shared tables.
Payment Service	Yes	Yes	Yes	Yes	Yes	No issue; owns payment state and hides provider details.
Notification Service	Yes	Yes	Yes	Yes	Yes	No issue; owns notification delivery state.